F

实验：教程

这是一份非常简短的文档，帮你熟悉 UNIX 系统上 C 编程环境的基础知识。它不全面，也不特别详细，但应该足以让你入门。

关于编程，有几条一般性的建议：如果你想成为编程高手，光掌握一门语言的语法是不够的。具体来说，你应该**了解你的工具**、**了解你的库**、**了解你的文档**。和 C 编译相关的工具有 **gcc**、**gdb**，也许还有 **ld**。你还可以用到大量的库例程，好在很多功能都包含在 **libc** 里，所有 C 程序默认都会链接它，你只需要包含正确的头文件就行。最后，知道怎样找到你需要的库例程（例如，学会查找和阅读手册页）是一项值得掌握的技能。后面我们会更详细地逐一介绍这些内容。

和人生中（几乎）所有值得做的事一样，要在这些方面成为高手需要时间。花点时间事先多了解一些工具和环境，绝对是值得的。

F.1 一个简单的 C 程序

我们从一个简单的 C 程序开始，比如说把它保存在文件“hw.c”里。和 Java 不同，文件名和文件内容之间不一定有关联；所以，用常识给文件起个合适的名字就行。

第一行指定要包含的文件，这里是 `stdio.h`，它给出了许多常用输入/输出例程的“原型”，我们关心的是其中的 `printf()`。使用 `#include` 指令，就是告诉 C 预处理器（`cpp`）找到某个文件（例如 `stdio.h`），并把它直接插入到代码中 `#include` 所在的位置。默认情况下，`cpp` 会到 `/usr/include/` 目录下去找这个文件。

接下来指定 `main()` 例程的签名：它返回一个整数（`int`），调用时会传入两个参数：一个整数 `argc`，它是命令行参数的个数；一个字符指针数组（`argv`），其中每个指针指向命令行中的一个词，最后一个是空指针。下面还会进一步介绍指针和数组。

然后，程序只是打印字符串“hello, world”，并让输出换到下一行，这要归功于 `printf()` 调用末尾的反斜杠加“n”。之后，程序返回一个值并结束，这个值会传回给执行该程序的 shell。脚本或终端前的用户可以检查这个值（在 csh 和 tcsh 中，它保存在 `status` 变量里），看看程序是正常退出还是出错退出。

F.2 编译和执行

现在我们来学习怎样编译这个程序。注意，我们用 `gcc` 作例子，不过在有些平台上，你也可以用另一个（平台自带的）编译器 `cc`。

```
/* 头文件放在这上面 */
/* 注意，C 的注释用斜杠和星号括起来，
   可以跨行 */
// 两个斜杠也可以（而且可能更好）
#include <stdio.h>

// main 返回一个整数
int main(int argc, char *argv[]) {
    /* printf 是我们的输出函数；
       默认写到标准输出 */
    /* printf 会返回一个整数，但我们忽略它 */
    printf("hello, world\n");

    /* 返回 0，表示一切顺利 */
    return(0);
}
```

在 shell 提示符下，只需输入：

```
prompt> gcc hw.c
```

gcc 其实并不是编译器，而是一个叫作“编译器驱动程序”（compiler driver）的程序，它负责协调编译的许多步骤，通常有四五步。首先，gcc 会执行 C 预处理器 cpp，处理某些指令（比如 #define 和 #include）。cpp 只是一个源码到源码的翻译器，所以它的产物仍然是源代码（即一个 C 文件）。然后真正的编译开始，通常由一个叫 cc1 的命令完成，它把源码级的 C 代码转换成和本机相关的底层汇编代码。接着会执行汇编器 as，生成目标代码（机器真正能理解的二进制位之类的东西）。最后，由链接编辑器（也就是链接器）ld 把所有东西组合成最终的可执行程序。幸运的是（!），在大多数情况下，你完全可以不管 gcc 是怎么工作的，只要带上合适的选项使用它就行。

上面编译的结果是一个可执行文件，（默认）名为 a.out。要运行这个程序，只需输入：

```
prompt> ./a.out
```

运行这个程序时，操作系统会正确设置 argc 和 argv，好让程序按需处理命令行参数。具体来说，argc 等于 1，argv[0] 是字符串“./a.out”，argv[1] 是空指针，表示数组结束。

F.3 有用的选项

在讲 C 语言之前，我们先指出 gcc 的一些有用的编译选项。

```
prompt> gcc -o hw hw.c    # -o：指定可执行文件的名字
prompt> gcc -Wall hw.c    # -Wall：给出好得多的警告
prompt> gcc -g hw.c       # -g：启用 gdb 调试
prompt> gcc -O hw.c       # -O：打开优化
```

当然，你可以按需组合这些选项（例如 gcc -o hw -g -Wall hw.c）。在这些选项中，你应该始终使用 -Wall，它会针对可能的错误给出大量额外的警告。**不要忽略这些警告！**而是要把问题改掉，让它们统统消失，心里也就踏实了。

F.4 链接库

有时，你可能想在程序中使用某个库例程。由于 C 库里有大量例程（每个程序都会自动链接 C 库），通常你只需找到正确的 `#include` 文件。最好的办法是查**手册页**（manual pages），通常就叫 **man 页**（man pages）。

例如，假设你想用 `fork()` 系统调用[1]。在 shell 提示符下输入 `man fork`，你会得到一段文字，介绍 `fork()` 是怎么工作的。最上面是一小段代码，告诉你程序中需要 `#include` 哪些文件才能通过编译。对于 `fork()`，你需要 `#include` 文件 `unistd.h`，写法如下：

```
#include <unistd.h>
```

不过，有些库例程不在 C 库里，你就得多做一点工作。例如，数学库里有很多有用的例程，比如正弦、余弦、正切等。如果你想在代码中使用例程 `tan()`，同样应该先查手册页。在 Linux 上 `tan` 的手册页最上面，你会看到下面两行：

```
#include <math.h>
...
Link with -lm.
```

第一行你应该已经明白了：你需要 `#include` 数学库，它位于文件系统中的标准位置（即 `/usr/include/math.h`）。而下一行告诉你的是怎样把程序和数学库“链接”起来。有许多有用的库可以链接，其中很多都放在 `/usr/lib` 中，数学库也确实在这里。

库有两种：静态链接库（以 `.a` 结尾）和动态链接库（以 `.so` 结尾）。静态链接库会直接合并到可执行文件中；也就是说，链接器把库的底层代码插入到你的可执行文件里，生成的二进制目标文件会大得多。动态链接对此做了改进，它只在程序的可执行文件中包含对库的引用；程序运行时，由操作系统的加载程序动态地把库链接进来。这种方法比静态方法更受青睐，因为它节省磁盘空间（不会生成不必要的大可执行文件），还能让应用程序在内存中共享库代码和静态数据。数学库既有静态版本，也有动态版本，静态版本叫 `/usr/lib/libm.a`，动态版本叫 `/usr/lib/libm.so`。

无论如何，要链接数学库，你都需要把这个库告诉链接编辑器；这可以通过给 `gcc` 加上合适的选项来实现。

```
prompt> gcc -o hw hw.c -Wall -lm
```

`-lXXX` 选项告诉链接器去找 `libXXX.so` 或 `libXXX.a`，大概就是按这个顺序找。如果出于某种原因，你坚持要用静态库而不是动态库，还有另一个选项可以用，看看你能不能自己查出来是哪个。人们有时更喜欢库的静态版本，因为使用动态库会带来一点性能开销。

最后再说一点：如果你想让编译器到通常位置以外的路径下找头文件，或者想让它链接你指定的库，可以用编译器选项 `-I/foo/bar` 到目录 `/foo/bar` 中查找头文件，用 `-L/foo/bar` 选项到 `/foo/bar` 目录中查找库。用这种方式指定的一个常见目录是“.”（读作“dot”），它是 UNIX 中当前目录的简写。注意，`-I` 选项应该放在编译命令行上，`-L` 选项应该放在链接命令行上。

[1]注意，`fork()` 是一个系统调用，而不只是一个库例程。不过，C 库为所有系统调用都提供了 C 包装函数，每个包装函数只是陷入操作系统而已。

F.5 分别编译

程序大到一定程度后，你可能想把它拆成几个文件，分别编译，然后再链接到一起。例如，假设你有两个文件 hw.c 和 helper.c，想分别编译它们，再链接在一起。

```
# 我们用 -Wall 打开警告，用 -O 打开优化
prompt> gcc -Wall -O -c hw.c
prompt> gcc -Wall -O -c helper.c
prompt> gcc -o hw hw.o helper.o -lm
```

-c 选项告诉编译器只生成目标文件，这里就是 hw.o 和 helper.o 两个文件。这些文件不是可执行文件，只是各个源文件中代码的机器级表示。要把目标文件组合成可执行文件，必须把它们“链接”到一起；这由第三行 gcc -o hw hw.o helper.o 完成。这时，gcc 发现指定的输入文件不是源文件（.c），而是目标文件（.o），于是直接跳到最后一步，调用链接编辑器 ld 把它们链接成一个可执行文件。由于它的作用，这一行通常叫作“链接行”，-lm 之类和链接相关的命令就应该写在这里。同理，-Wall 和 -O 之类的选项只在编译阶段需要，因此不必写在链接行上，只写在编译行上就行。

当然，你也可以把所有 C 源文件都写在一行里交给 gcc：

```
prompt> gcc -Wall -O -o hw hw.c helper.c
```

但这样系统就要重新编译每个源文件，可能很费时间。分别编译各个文件，就可以只重新编译你编辑时改动过的那些文件，从而节省时间、提高效率。这个过程最好交给另一个程序 make 来管理，下面就介绍它。

F.6 Makefile

程序 make 可以让构建过程的大部分工作自动化，因此对任何正经的程序（和程序员）来说都是极其重要的工具。我们来看一个简单的例子，它保存在名为 Makefile 的文件中。

```
hw: hw.o helper.o
    gcc -o hw hw.o helper.o -lm

hw.o: hw.c
    gcc -O -Wall -c hw.c

helper.o: helper.c
    gcc -O -Wall -c helper.c

clean:
    rm -f hw.o helper.o hw
```

现在，要构建程序，只需在命令行输入 make。它（默认）会查找 Makefile 或 makefile，并把它作为输入（你也可以用某个选项指定别的 makefile，读读手册页就知道是哪个）。GNU 版本的 make，即 gmake，功能比传统的 make 更完整，所以下面的讨论主要以它为准（不过这两个说法我们会混用）。下面的内容大多来自 gmake 的 info 页面；怎

样找到这些页面，请看后面的“文档”一节。另外请注意：在 Linux 系统上，gmake 和 make 是同一个东西。

Makefile 建立在规则（rule）之上，规则用来决定需要做什么。规则的一般形式如下：

```
target: prerequisite1 prerequisite2 ...
    command1
    command2
    ...
```

目标（target）通常是由命令生成的文件的名字，例如可执行文件或目标文件。目标也可以是要执行的某个动作的名字，比如我们例子中的“clean”。

依赖（prerequisite）是用来生成目标的输入文件。一个目标往往依赖好几个文件。例如，要构建可执行文件 hw，需要先构建两个目标文件：hw.o 和 helper.o。

最后，**命令**（command）是 make 要执行的动作。一条规则可以有多条命令，每条占一行。**注意：**每个命令行的开头都必须放一个制表符（tab）！如果你只放了空格，make 就会打印一条莫名其妙的错误信息，然后退出。

命令通常出现在带依赖的规则里，用来在任何一个依赖发生变化时生成目标文件。不过，为目标指定命令的规则不一定要有依赖。例如，我们例子中和目标“clean”对应、包含删除命令的那条规则就没有依赖。

回到我们的例子，执行 make 时，它大致是这样工作的：首先，它来到目标 hw，发现要构建它，必须先有两个依赖 hw.o 和 helper.o。也就是说，hw 依赖这两个目标文件。接着，make 会逐个检查这些目标。检查 hw.o 时，它会发现 hw.o 依赖 hw.c。关键就在这里：如果 hw.c 的修改时间比 hw.o 的生成时间更晚，make 就知道 hw.o 已经过时，应该重新生成；这时它会执行命令行 gcc -O -Wall -c hw.c，生成 hw.o。因此，如果你编译的是一个大程序，make 会根据依赖关系知道哪些目标文件需要重新生成，并且只做重新生成可执行文件所必需的工作。还要注意，如果 hw.o 根本不存在，也会生成它。

接下来，helper.o 也可能按照上面同样的标准被重新生成或新建。两个目标文件都生成之后，make 就可以执行生成最终可执行文件的命令了，于是它回过头去执行：gcc -o hw hw.o helper.o -lm。

到目前为止，我们一直忽略了 makefile 中的 clean 目标。要用它，必须显式地要求。输入

```
prompt> make clean
```

就会执行这条命令行上的命令。因为 clean 目标没有依赖，所以输入 make clean 总会执行其中的命令。这里 clean 目标用来删除目标文件和可执行文件，如果你想从头重新构建整个程序，这就非常方便。

现在你可能会想：“嗯，看起来还行，但这些 makefile 写起来也太麻烦了！”要是它们总得这么写，你说得没错。好在有很多简便写法能让 make 用起来更省事。例如，下面这个 makefile 功能相同，但用起来要方便一些：

```
# 在这里指定所有源文件
SRCS = hw.c helper.c
```

```
# 在这里指定目标（可执行文件的名字）
TARG = hw

# 指定编译器、编译选项和需要的库
CC   = gcc
OPTS = -Wall -O
LIBS = -lm

# 把源文件列表中的 .c 文件换成 .o 文件
OBJS = $(SRCS:.c=.o)

# all 其实不是必需的，但可以用来生成目标
all: $(TARG)

# 这条规则生成目标可执行文件
$(TARG): $(OBJS)
    $(CC) -o $(TARG) $(OBJS) $(LIBS)

# 这是 .o 文件的通用规则
%.o: %.c
    $(CC) $(OPTS) -c $< -o $@

# 最后是 clean 规则
clean:
    rm -f $(OBJS) $(TARG)
```

虽然我们不打算细讲 `make` 的语法，但你可以看到，这个 makefile 能让你的生活轻松一些。例如，要往构建中添加新的源文件，只需把它们加到 makefile 开头的 `SRCS` 变量里。改一下 `TARG` 那一行，就能轻松更换可执行文件的名字；编译器、选项和库的设置也都很容易修改。

关于 `make` 最后再说一句：弄清一个目标有哪些依赖并不总是那么简单，在大型复杂的程序中尤其如此。不出所料，有另一个工具可以帮忙，叫作 `makedepend`。自己去了解一下，看看能不能把它用到 makefile 里。

F.7 调试

终于，你建好了一个不错的构建环境，程序也正确编译了，却可能发现程序有 bug。解决问题的一种办法是冥思苦想，这种方法有时能成功，但往往不行。问题在于缺乏信息：你根本不知道程序内部到底发生了什么，所以也就想不出它为什么不按预期运行。好在有工具可以帮忙：GNU 调试器 `gdb`。

我们来看下面这段有 bug 的代码，它保存在文件 `buggy.c` 中，编译成了可执行文件 `buggy`。

```
#include <stdio.h>

struct Data {
    int x;
};

int
main(int argc, char *argv[])
{
    struct Data *p = NULL;
    printf("%d\n", p->x);
}
```

在这个例子中，主程序在变量 p 为 NULL 时对它解引用，这会导致段错误。当然，这个问题一眼就能看出来、很容易修好，但在更复杂的程序里，找到这样的问题并不总是那么容易。

为了准备调试，要重新编译程序，并确保在每个编译行上都加上 -g 选项。这会在可执行文件中加入额外的调试信息，在调试时很有用。另外，不要打开优化（-O）；虽然这样也许能行，但也可能在调试时把你弄糊涂。

用 -g 重新编译之后，就可以使用调试器了。在命令提示符下这样启动 gdb：

```
prompt> gdb buggy
```

这样你就进入了和调试器的交互会话。注意，你还可以用调试器检查程序异常运行时生成的“core”文件，或者附加到一个正在运行的程序上；要了解更多，请阅读文档。

进去之后，你可能会看到类似这样的内容：

```
prompt> gdb buggy
GNU gdb ...
Copyright 2008 Free Software Foundation, Inc.
(gdb)
```

你可能想做的第一件事就是运行程序。只需在 gdb 命令提示符下输入 run。在这个例子中，你可能会看到：

```
(gdb) run
Starting program: buggy

Program received signal SIGSEGV, Segmentation fault.
0x8048433 in main (argc=1, argv=0xbffff844) at buggy.c:19
19          printf("%d\n", p->x);
```

从这个例子可以看到，gdb 立刻指出了问题出在哪里：在我们试图解引用 p 的那一行产生了“段错误”。这其实就是说，我们访问了不该访问的内存。这时，机灵的程序员看一眼代码就会说：“啊哈！一定是 p 没有指向任何有效的东西，所以不该对它解引用！”然后就去把问题改掉。

不过，如果你不知道发生了什么，可能会想检查某个变量。gdb 允许你在调试会话中交互地做这件事。

```
(gdb) print p
1 = (Data *) 0x0
```

用 `print` 原语检查 `p`，可以看到它是一个指向 `Data` 类型结构体的指针，而且当前被设为 `NULL`（也就是零，或者这里显示的十六进制零“0x0”）。

最后，你还可以在程序中设置断点，让调试器在某个例程处把程序停下来。这之后，逐步执行程序（一次一行），看看发生了什么，往往很有用。

```
(gdb) break main
Breakpoint 1 at 0x8048426: file buggy.c, line 17.
(gdb) run
Starting program: /homes/hacker/buggy
Breakpoint 1, main (argc=1, argv=0xbffff844) at buggy.c:17
17        struct Data *p = NULL;
(gdb) next
19        printf("%d\n", p->x);
(gdb) next

Program received signal SIGSEGV, Segmentation fault.
0x8048433 in main (argc=1, argv=0xbffff844) at buggy.c:19
19        printf("%d\n", p->x);
```

在上面的例子中，我们在 `main()` 例程处设了一个断点；因此运行程序时，调试器几乎立刻就在 `main` 处停了下来。这时，我们在例子中发出了一条“next”命令，它会执行下一条源码级语句。“next”和“step”都是推进程序执行的有用方式，详情请阅读文档[2]。

这里的讨论远远没有把 `gdb` 讲透；它是一个功能丰富、用法灵活的调试工具，功能远比这点篇幅里能介绍的多。请在你大把的空闲时间里自己多读一读，成为调试高手吧。

F.8 文档

要深入了解所有这些东西，你得做两件事：一是用这些工具，二是自己多读相关资料。要进一步了解 `gcc`、`gmake` 和 `gdb`，一个办法是读它们的手册页：在命令提示符下输入 `man gcc`、`man gmake` 或 `man gdb`。你还可以用 `man -k` 按关键词搜索手册页，不过效果不一定总是理想；这时上网搜一下可能是更好的办法。

手册页有一个麻烦的地方：如果有不止一个叫 XXX 的东西，输入 `man XXX` 不一定能得到你想要的那个。例如，如果你要找 `kill()` 系统调用的手册页，只在提示符下输入 `man kill`，就会得到错误的手册页，因为还有一个叫 `kill` 的命令行程序。手册页分成若干**节**（section），默认情况下，`man` 会返回它找到的编号最小的那一节中的手册页，在这个例子中就是第 1 节。注意，看一下页面顶部就能知道你拿到的是哪个手册页：如果看到 `kill(2)`，就说明你找对了，是第 2 节的手册页，系统调用就在这一节。输入 `man man` 可以进一步了解手册页的各节分别存放什么内容。另外，`man -a kill` 可以依次查看所有名叫“kill”的手册页。

[2]特别是，用 gdb 调试时，你可以使用交互式的“help”命令。

手册页能帮你查到很多东西。特别是，你常常需要查一个库调用该传哪些参数，或者使用某个库调用需要包含哪些头文件。这些都应该能在手册页里找到。例如，查一下 open() 系统调用，你会看到：

```
SYNOPSIS
    #include <sys/types.h>
    #include <sys/stat.h>
    #include <fcntl.h>

    int open(const char *path, int oflag,
             /* mode_t mode */...);
```

这告诉你，要使用 open 调用，需要包含头文件 sys/types.h、sys/stat.h 和 fcntl.h。它还告诉你要传给 open 的参数：一个名为 path 的字符串、一个整数标志 oflag，以及一个可选参数，用来指定文件的模式。如果使用这个调用还需要链接某些库，这里也会告诉你。

手册页需要花点功夫才能用好。它们通常分成若干标准部分。正文部分会说明怎样传入不同的参数，让函数表现出不同的行为。

一个特别有用的部分叫作 RETURN VALUES，它告诉你函数在成功或失败时会返回什么。还是看 open() 的手册页：

```
RETURN VALUES
    成功完成时，open() 函数打开文件，并返回一个非负整数，
    即编号最小的未使用的文件描述符。否则返回 -1，并设置
    errno 来指明错误，而且不会创建或修改任何文件。
```

因此，检查 open 的返回值，就能知道打开是否成功。如果没成功，open（以及许多标准库例程）会把一个叫 errno 的全局变量设为某个值，告诉你出了什么错。详情请看手册页的 ERRORS 部分。

你可能还想做另一件事：查找手册页里没有给出的某个结构体的定义。例如，gettimeofday() 的手册页有如下概要：

```
SYNOPSIS
    #include <sys/time.h>

    int gettimeofday(struct timeval *restrict tp,
                     void *restrict tzp);
```

从这一页可以看到，时间放在一个 timeval 类型的结构体里，但手册页不一定会告诉你这个结构体有哪些字段！（这次它告诉了你，但你不一定总有这么好的运气。）所以，你可能得自己去找。所有的头文件都在 /usr/include 目录下，因此可以用 grep 这样的工具来找。例如，你可以输入：

```
prompt> grep 'struct timeval' /usr/include/sys/*.h
```

这样就能在 /usr/include/sys 下所有以 .h 结尾的文件中查找这个结构体的定义。遗憾的是，这样做不一定总能找到，因为这个头文件可能又包含了放在别处的其他头文件。

更好的办法是用你手头的一个工具：编译器。写一个包含头文件 `time.h` 的程序，比如叫 `main.c`。然后不编译它，而是用编译器调用预处理器。预处理器会处理文件中的所有指令，比如 `#define` 命令和 `#include` 命令。为此，输入 `gcc -E main.c`。结果是一个 C 文件，里面有所有需要的结构体和原型，包括 `timeval` 结构体的定义。

要查这些东西，也许还有更好的办法：上网搜。不知道的东西就应该去搜一搜。只要查一查就能学到这么多东西，真是太神奇了！

info 页面

在查找文档时，**info 页面**（info pages）也非常有用，它为许多 GNU 工具提供了详细得多的文档。你可以运行 `info` 程序来访问 info 页面，也可以在黑客们偏爱的编辑器 emacs 里执行 `Meta-x info` 来访问。像 `gcc` 这样的程序有上百个选项，其中一些知道了会出乎意料地有用。`gmake` 还有许多功能，能改善你的构建环境。最后，`gdb` 是一个相当精巧的调试器。读一读手册页和 info 页面，试试以前没用过的功能，成为善用编程工具的高手吧。

F.9 推荐阅读

除了手册页和 info 页面，还有很多有用的书。注意，这些信息很多都能在网上免费找到；不过，书的形式有时似乎更便于学习。另外，对你感兴趣的主题，一定要找找 O'Reilly 出版的书；它们几乎总是质量上乘。

- "The C Programming Language"，作者 Brian Kernighan 和 Dennis Ritchie。这是那本必备的权威 C 语言书。
- "Managing Projects with make"，作者 Andrew Oram 和 Steve Talbott。一本讲 `make` 的书，篇幅不长，内容中肯。
- "Debugging with GDB: The GNU Source-Level Debugger"，作者 Richard M. Stallman 和 Roland H. Pesch。一本讲怎样使用 GDB 的小书。
- "Advanced Programming in the UNIX Environment"，作者 W. Richard Stevens 和 Steve Rago。Stevens 写过几本非常好的书，这本是 UNIX 黑客的必读书。他还写了一套讲 TCP/IP 和套接字编程的好书。
- "Expert C Programming"，作者 Peter Van der Linden。上面关于编译器等方面的很多有用技巧，都是直接从这本书里"偷"来的。读读它！虽然有点过时，但这是一本很棒的书，能让你大开眼界。

 © 2008–23, ARPACI-DUSSEAU